TripCraft — Complete Solution

> The whole supervisor-led team, every file — nothing to copy from elsewhere. Use it to **check your work**, not to skip the thinking. Build from the story first; read the hints if you get stuck.

The final layout

```
<repo folder>/
├── data/
│   └── attractions.json
├── src/
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── analysis_agent.py
│   │   ├── research_agent.py
│   │   ├── reviewer_agent.py
│   │   ├── supervisor.py
│   │   └── writer_agent.py
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py
│   ├── prompts/
│   │   ├── analysis.txt
│   │   ├── reviewer.txt
│   │   └── writer.txt
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── logger.py
│   │   └── prompts.py
│   ├── __init__.py
│   ├── config.py
│   ├── exceptions.py
│   ├── graph.py
│   ├── schemas.py
│   └── state.py
├── tests/
│   ├── integration/
│   │   └── test_live_team.py
│   └── unit/
│       ├── test_config.py
```

```
│       ├── test_lookup.py
│       └── test_supervisor.py
├── .env.example
├── .gitignore
├── LICENSE
├── main.py
└── requirements.txt
```

Setup files

`requirements.txt`

```
langchain-core>=0.3
langchain-openai>=0.2
langgraph>=0.2
pydantic>=2.0
python-dotenv>=1.0
```

`.env.example` (copy to `.env`)

```
OPENAI_API_KEY=sk-your-key-here
LOG_LEVEL=INFO
```

`.gitignore`

```
.env
.venv/
__pycache__/
*.pyc
```

Every folder under `src/` gets an empty `__init__.py`. `LICENSE` is the MIT text with your name and the year.

The shared state and the review verdict

```
# src/state.py
# The one shared state the whole team reads and writes. One writer per
# field: research writes notes, analysis writes day_plan, writer writes
# draft, reviewer writes score+feedback, the supervisor writes routing_log.
from typing import TypedDict
```

```
class TeamState(TypedDict):
    destination: str
    notes: list          # attractions, each ending with a [source] tag
    day_plan: str
    draft: str
    score: int
    feedback: str
    revisions: int
    routing_log: list
    status: str          # "working" | "done" | "unknown" | "no_agreement"

def start_state(destination: str) -> TeamState:
    return {
        "destination": destination,
        "notes": None,
        "day_plan": None,
        "draft": None,
        "score": None,
        "feedback": "",
        "revisions": 0,
        "routing_log": [],
        "status": "working",
    }
```

```
# src/schemas.py
# The reviewer's checked verdict — a real number the code can compare with
# PASS_SCORE, never free text to grep.
from pydantic import BaseModel, Field

class ReviewVerdict(BaseModel):
    score: int = Field(ge=1, le=10)
    feedback: str
```

Settings, errors, logging, prompts

```
# src/exceptions.py
class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
```

```
from dotenv import load_dotenv
from src.exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
    log_level: str

def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config() -> Config:
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
    )
```

```
# src/utils/logger.py
import logging
import os

def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/utils/prompts.py

def load_prompt(name: str) -> str:
    with open("src/prompts" + "/" + name + ".txt") as prompt_file:
        return prompt_file.read().strip()
```

```
# src/models/llm.py
from langchain_openai import ChatOpenAI

from src.config import Config

def get_model(config: Config) -> ChatOpenAI:
    return ChatOpenAI(
        model="gpt-4o-mini", temperature=0, api_key=config.openai_api_key
    )
```

The research tool and its data

```
# data/attractions.json
{
  "paris": [
    "Eiffel Tower — iron-lattice tower, city views [source: paris.gov]",
    "Louvre Museum — the Mona Lisa and more [source: louvre.fr]",
    "Notre-Dame — gothic cathedral on the Seine [source: paris.gov]",
    "Montmartre — hilltop art district [source: paris.gov]",
    "Musee d'Orsay — impressionist art [source: orsay.fr]"
  ],
  "tokyo": [
    "Senso-ji — historic temple in Asakusa [source: tokyo.go.jp]",
    "Shibuya Crossing — famous scramble crossing [source: tokyo.go.jp]",
    "Meiji Shrine — forested shrine [source: tokyo.go.jp]",
    "Tokyo Skytree — tall broadcasting tower [source: skytree.jp]"
  ]
}
```

```
# src/agents/research_agent.py
# The Research specialist: looks attractions up from the local file — never
# from the model's memory. Writes the notes field; nothing else.
import json

from langchain_core.tools import tool

from src.utils.logger import get_logger

@tool
def lookup_attractions(destination: str) -> list:
    """Look up real attractions for a destination from the local file."""
    with open("data/attractions.json") as attractions_file:
        data = json.loads(attractions_file.read())
```

```
    return data.get(destination.strip().lower(), [])

def run_research(destination: str) -> list:
    logger = get_logger(__name__)
    found = lookup_attractions.invoke({"destination": destination})
    logger.info(f"research found {len(found)} attractions for {destination}")
    return found
```

```
# src/prompts/analysis.txt
You group attractions into 2-4 themed days for a short trip.
Keep EVERY attraction and its [source] tag. Do not invent any.
Return a short plan: "Day 1 (theme): ...", one line per day.

Attractions:
{notes}
```

```
# src/agents/analysis_agent.py
# The Analysis specialist (the planner): groups the looked-up attractions
# into themed days, keeping every one. Writes the day_plan field.
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

from src.utils.logger import get_logger
from src.utils.prompts import load_prompt

def run_analysis(model, notes: list) -> str:
    logger = get_logger(__name__)
    prompt = ChatPromptTemplate.from_template(load_prompt("analysis"))
    chain = prompt | model | StrOutputParser()
    joined = "\n".join(notes)
    plan = chain.invoke({"notes": joined})
    logger.info("analysis produced a day plan")
    return plan
```

```
# src/prompts/writer.txt
You write a short day-by-day travel itinerary from the day plan ONLY.
Use only attractions named in the plan, and keep every [source] tag.
If there is reviewer feedback, address it. Keep it concise.

Day plan:
{day_plan}

Reviewer feedback (may be empty):
{feedback}
```

```
# src/agents/writer_agent.py
# The Writer specialist: writes the itinerary from the plan only. Writes the
# draft field.
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

from src.utils.logger import get_logger
from src.utils.prompts import load_prompt

def run_writer(model, day_plan: str, feedback: str) -> str:
    logger = get_logger(__name__)
    prompt = ChatPromptTemplate.from_template(load_prompt("writer"))
    chain = prompt | model | StrOutputParser()
    draft = chain.invoke({"day_plan": day_plan, "feedback": feedback})
    logger.info("writer produced a draft itinerary")
    return draft
```

```
# src/prompts/reviewer.txt
You review a travel itinerary. Score it 1-10 on: realistic pacing,
grounded in the sourced attractions, covers the main ones. Give one or
two sentences of feedback naming anything to fix.

Attractions (the sources of truth):
{notes}

Itinerary to review:
{draft}
```

```
# src/agents/reviewer_agent.py
# The Reviewer specialist: returns a checked ReviewVerdict (score+feedback).
# The pass mark lives in code (PASS_SCORE), not in the prompt.
from langchain_core.prompts import ChatPromptTemplate

from src.schemas import ReviewVerdict
from src.utils.logger import get_logger
from src.utils.prompts import load_prompt

PASS_SCORE = 7

def run_reviewer(model, notes: list, draft: str) -> ReviewVerdict:
    logger = get_logger(__name__)
    prompt = ChatPromptTemplate.from_template(load_prompt("reviewer"))
    chain = prompt | model.with_structured_output(ReviewVerdict)
    verdict = chain.invoke({"notes": "\n".join(notes), "draft": draft})
```

```
    logger.info(f"reviewer scored {verdict.score}")
    return verdict
```

The supervisor and the nodes

```
# src/agents/supervisor.py
# The manager. decide_next() reads the shared state and names who works next
# and why; supervisor() logs that decision and routes with Command. The
# supervisor is the ONLY writer of routing_log.
from langgraph.graph import END
from langgraph.types import Command

from src.agents.reviewer_agent import PASS_SCORE
from src.utils.logger import get_logger

MAX_REVISIONS = 2

def decide_next(state) -> tuple:
    if state["notes"] is None:
        return "research", "no attractions looked up yet"
    if len(state["notes"]) == 0:
        return "finish_unknown", "nothing known about this destination"
    if state["day_plan"] is None:
        return "analysis", "attractions found, need a day plan"
    if state["draft"] is None:
        return "writer", "have a plan, need an itinerary"
    if state["score"] is None:
        return "reviewer", "have a draft, need a review"
    if state["score"] >= PASS_SCORE:
        return "finish_ok", f"accepted with score {state['score']}"
    if state["revisions"] >= MAX_REVISIONS:
        return "finish_stuck", "writer and reviewer could not agree"
    return "revise", f"rejected ({state['score']}), sending back"

def supervisor(state):
    logger = get_logger(__name__)
    target, reason = decide_next(state)
    log = state["routing_log"] + [{"step": target, "reason": reason}]
    logger.info(f"supervisor -> {target}: {reason}")

    if target == "finish_ok":
        return Command(goto=END, update={"routing_log": log, "status": "done"})
    if target == "finish_unknown":
        return Command(goto=END,
                       update={"routing_log": log, "status": "unknown"})
```

```
    if target == "finish_stuck":
        return Command(goto=END,
                       update={"routing_log": log, "status": "no_agreement"})
    if target == "revise":
        # clear the score and bump the count, then let the writer run again
        return Command(goto="writer", update={
            "routing_log": log,
            "score": None,
            "revisions": state["revisions"] + 1,
        })
    return Command(goto=target, update={"routing_log": log})
```

```
# src/graph.py
# The LangGraph graph: five working nodes plus the supervisor, one edge
# (START -> supervisor). Every other move is a Command a node returns.
from langgraph.graph import START, StateGraph
from langgraph.types import Command

from src.agents.analysis_agent import run_analysis
from src.agents.research_agent import run_research
from src.agents.reviewer_agent import run_reviewer
from src.agents.supervisor import supervisor
from src.agents.writer_agent import run_writer
from src.state import TeamState

def build_graph(model):
    def research_node(state):
        notes = run_research(state["destination"])
        return Command(goto="supervisor", update={"notes": notes})

    def analysis_node(state):
        plan = run_analysis(model, state["notes"])
        return Command(goto="supervisor", update={"day_plan": plan})

    def writer_node(state):
        draft = run_writer(model, state["day_plan"], state["feedback"])
        return Command(goto="supervisor", update={"draft": draft})

    def reviewer_node(state):
        verdict = run_reviewer(model, state["notes"], state["draft"])
        return Command(goto="supervisor", update={
            "score": verdict.score, "feedback": verdict.feedback,
        })

    graph = StateGraph(TeamState)
    graph.add_node("supervisor", supervisor)
```

```
    graph.add_node("research", research_node)
    graph.add_node("analysis", analysis_node)
    graph.add_node("writer", writer_node)
    graph.add_node("reviewer", reviewer_node)
    graph.add_edge(START, "supervisor")
    return graph.compile()
```

The entry point

```
# main.py
# Type a destination, get a reviewed itinerary (or an honest stop), plus the
# routing log showing who ran and why.
import sys

from src.config import load_config
from src.exceptions import MissingConfigError
from src.graph import build_graph
from src.models.llm import get_model
from src.state import start_state

def print_result(final):
    print("\nRouting log:")
    for entry in final["routing_log"]:
        print(f"  {entry['step']}: {entry['reason']}")
    if final["status"] == "done":
        print("\nItinerary:\n" + final["draft"])
    elif final["status"] == "unknown":
        print("\nNothing is known about this destination yet.")
    else:
        print("\nThe team could not agree on an itinerary.")

def main():
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    model = get_model(config)
    app = build_graph(model)
    print("TripCraft — type a destination, or /quit to exit.")
    while True:
        destination = input("Destination: ").strip()
        if destination == "/quit":
            break
```

```
        if destination == "":
            continue
        final = app.invoke(start_state(destination))
        print_result(final)

if __name__ == "__main__":
    main()
```

The tests

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "TRIPCRAFT_TEST_SETTING"

def test_missing_setting() -> None:
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")

def test_empty_setting() -> None:
    os.environ[TEST_SETTING] = ""
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 2 (empty setting)   -> {result}")

def test_config_loads() -> None:
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 3 (config loads)    -> {result}")
```

```
if __name__ == "__main__":
    test_missing_setting()
    test_empty_setting()
    test_config_loads()
```

```
# tests/unit/test_lookup.py
# The research tool, no model, no network.
# Run from the repo root: python -m tests.unit.test_lookup
from src.agents.research_agent import lookup_attractions

def test_known_destination() -> None:
    found = lookup_attractions.invoke({"destination": "Paris"})
    result = "OK"
    if len(found) == 0:
        result = "WRONG"
    if "[source" not in found[0]:
        result = "WRONG"
    print(f"case 1 (known destination has sources) -> {result}")

def test_case_insensitive() -> None:
    found = lookup_attractions.invoke({"destination": "  TOKYO "})
    result = "OK"
    if len(found) == 0:
        result = "WRONG"
    print(f"case 2 (trim + lowercase match)        -> {result}")

def test_unknown_destination() -> None:
    found = lookup_attractions.invoke({"destination": "Atlantis"})
    result = "OK"
    if found != []:
        result = "WRONG"
    print(f"case 3 (unknown returns empty)         -> {result}")

if __name__ == "__main__":
    test_known_destination()
    test_case_insensitive()
    test_unknown_destination()
```

```
# tests/unit/test_supervisor.py
# The routing brain, on hand-made states — no model, no network.
# Run from the repo root: python -m tests.unit.test_supervisor
from src.agents.research_agent import run_research  # noqa: F401
from src.agents.supervisor import MAX_REVISIONS, decide_next
from src.state import start_state
```

```
def test_routes_to_research_first() -> None:
    target, reason = decide_next(start_state("Paris"))
    print(f"case 1 (fresh -> research) -> "
          f"{'OK' if target == 'research' else 'WRONG'}")

def test_unknown_destination_stops() -> None:
    state = start_state("Atlantis")
    state["notes"] = []
    target, reason = decide_next(state)
    print(f"case 2 (empty notes -> finish_unknown) -> "
          f"{'OK' if target == 'finish_unknown' else 'WRONG'}")

def test_accepts_high_score() -> None:
    state = start_state("Paris")
    state["notes"] = ["x [source: a]"]
    state["day_plan"] = "Day 1"
    state["draft"] = "itinerary"
    state["score"] = 9
    target, reason = decide_next(state)
    print(f"case 3 (score 9 -> finish_ok) -> "
          f"{'OK' if target == 'finish_ok' else 'WRONG'}")

def test_revise_then_give_up() -> None:
    state = start_state("Paris")
    state["notes"] = ["x [source: a]"]
    state["day_plan"] = "Day 1"
    state["draft"] = "itinerary"
    state["score"] = 4
    state["revisions"] = 1
    r1 = decide_next(state)[0]
    state["revisions"] = MAX_REVISIONS
    r2 = decide_next(state)[0]
    ok = r1 == "revise" and r2 == "finish_stuck"
    print(f"case 4 (reject -> revise, then give up) -> "
          f"{'OK' if ok else 'WRONG'}")

if __name__ == "__main__":
    test_routes_to_research_first()
    test_unknown_destination_stops()
    test_accepts_high_score()
    test_revise_then_give_up()
```

```
# tests/integration/test_live_team.py
# The whole team against the real model. Run from the repo root:
#   python -m tests.integration.test_live_team
from src.config import load_config
```

```
from src.graph import build_graph
from src.models.llm import get_model
from src.state import start_state

def build_app():
    config = load_config()
    model = get_model(config)
    return build_graph(model)

def test_known_destination_finishes() -> None:
    final = build_app().invoke(start_state("Paris"))
    result = "OK"
    if final["status"] != "done":
        result = "WRONG"
    if "[source" not in final["draft"]:
        result = "WRONG"
    print(f"case 1 (Paris -> reviewed itinerary, sources kept) -> {result}")

def test_unknown_destination_stops() -> None:
    final = build_app().invoke(start_state("Atlantis"))
    result = "OK"
    if final["status"] != "unknown":
        result = "WRONG"
    print(f"case 2 (Atlantis -> honest stop)                  -> {result}")

if __name__ == "__main__":
    test_known_destination_finishes()
    test_unknown_destination_stops()
```

Expected unit output (all `OK`):

```
case 1 (missing setting) -> OK
case 2 (empty setting)   -> OK
case 3 (config loads)    -> OK
```

```
case 1 (known destination has sources) -> OK
case 2 (trim + lowercase match)        -> OK
case 3 (unknown returns empty)         -> OK
```

```
case 1 (fresh -> research) -> OK
case 2 (empty notes -> finish_unknown) -> OK
case 3 (score 9 -> finish_ok) -> OK
case 4 (reject -> revise, then give up) -> OK
```

That's the whole team. A known destination flows supervisor → research → analysis → writer → reviewer and comes back reviewed with its `[source]` tags; an unknown one stops cleanly; a draft the reviewer won't pass goes back to the writer, bounded by `MAX_REVISIONS`.